

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 4
Comparative Study of Deep Convolutional Neural Network
Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name & Reg. No.	Sadhumitha S.	24011101091

1. Objective

To study the evolution of deep convolutional neural network architectures, compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet, and to implement transfer learning by fine-tuning a pretrained CNN (ResNet50) on the CIFAR-10 dataset.

2. Learning Outcomes

Students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Background Theory

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The progression of CNN architectures began with LeNet-5 and evolved through AlexNet, VGGNet, GoogleNet, and ResNet, each introducing innovations that significantly improved image classification performance.

3.1 Evolution of CNN Architectures

Table 1: Evolution of CNN architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

LeNet-5. Proposed by Yann LeCun in 1998 for handwritten digit recognition, it consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

AlexNet. Won the ImageNet Challenge in 2012 by significantly reducing classification error. It introduced ReLU activation, Dropout regularization, GPU training, and data augmentation.

VGG16. Demonstrated that stacking multiple small 3×3 convolution filters could outperform larger filters while increasing network depth, giving excellent feature extraction with a simple, uniform architecture.

GoogleNet (Inception Network). Proposed by Szegedy et al. in 2014, it introduced the Inception Module, which performs 1×1 , 3×3 , 5×5 convolutions and max pooling in parallel and concatenates the outputs to obtain multi-scale feature representations. The 1×1 convolutions before larger kernels reduce the number of trainable parameters.

ResNet. Increasing CNN depth often causes the vanishing gradient problem, making optimization difficult. ResNet solves this using skip (residual) connections. Instead of learning $H(x)$ directly, ResNet learns the residual mapping

$$F(x) = H(x) - x, \quad \text{so that} \quad \text{Output} = F(x) + x$$

where $F(x)$ is the residual mapping and x is the identity mapping passed through the skip connection. This allows gradients to flow directly through the network, enabling very deep architectures with faster convergence and higher ImageNet accuracy.

Table 2: Comparison of architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

3.2 Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet, instead of training a network from scratch. The workflow used in this experiment is:

1. Load a pretrained model (ResNet50) with ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer and a new Dense classifier.
5. Train the new classifier on CIFAR-10.
6. Unfreeze the last few convolutional layers and fine-tune at a lower learning rate.

This gives faster convergence, higher accuracy on small datasets, reduced training time, and lower computational cost compared to training a deep CNN from scratch.

4. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

The ten classes present in the dataset are Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, and Truck. The dataset was loaded directly using `tensorflow.keras.datasets.cifar10`, and pixel values were normalized to the range $[0, 1]$ before being fed into the network.

5. Experimental Procedure

Task 1: Dataset Preparation

The CIFAR-10 dataset was loaded, normalized, and ten sample images were displayed along with their class labels to verify correct loading.



Figure 1: Sample images from the CIFAR-10 dataset with their class labels.

Task 2: Transfer Learning Model

A pretrained ResNet50 model (ImageNet weights, top classification layer removed) was used as the convolutional base. The base was frozen, and a Global Average Pooling layer, a Dense layer with ReLU activation (256 units), and a final Dense output layer with Softmax activation (10 classes) were added on top.

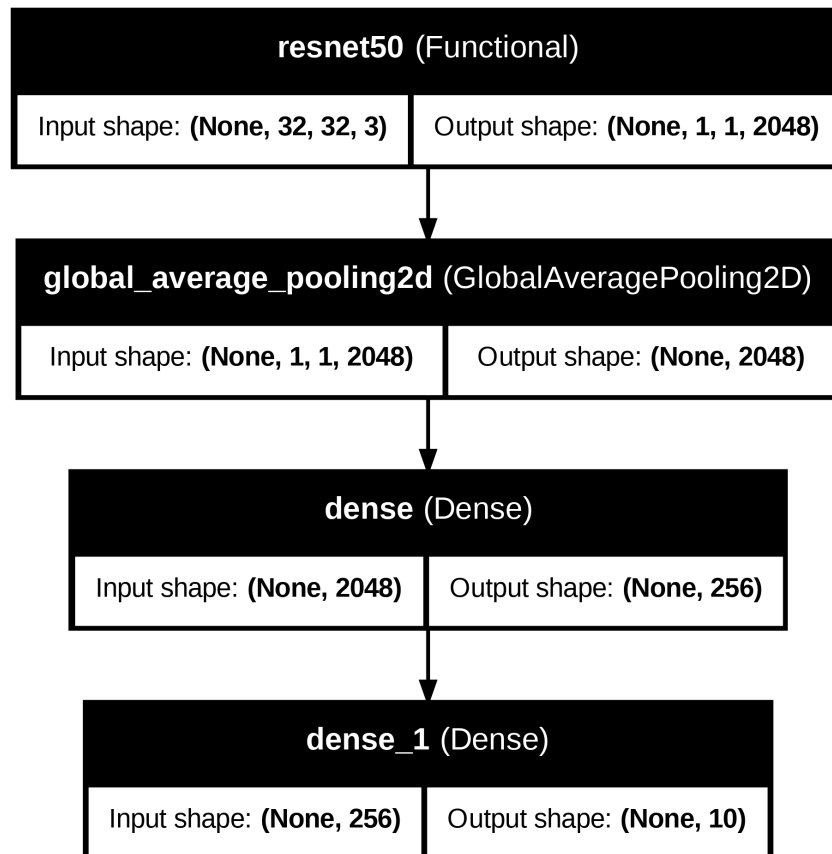


Figure 2: Transfer learning architecture built on top of the frozen ResNet50 base.

Task 3: Model Training

The classifier head was trained with the convolutional base frozen, using the configuration below.

Table 3: Training configuration

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10
Loss Function	Categorical Cross-Entropy
Evaluation Metric	Accuracy

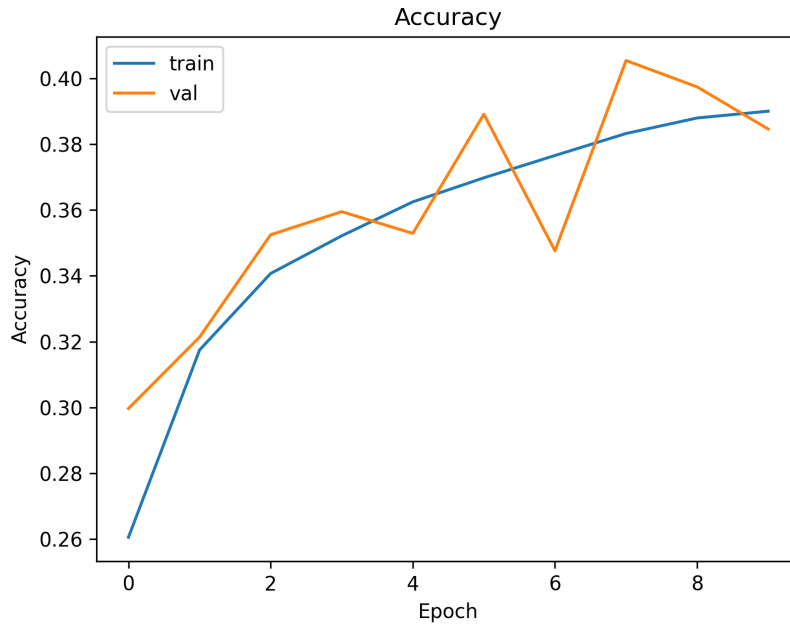


Figure 3: Training and validation accuracy over epochs (frozen base).

Inference: The training accuracy increased gradually from 26.06% in epoch 1 to 39.00% in epoch 10, while the validation accuracy increased from 29.97% to a high of 40.54% (epoch 8) and ended up at 38.46% in epoch 10. The two curves remained very close, with the validation accuracy closely following the training accuracy and even being higher than the training accuracy in some epochs, which shows that there was no overfitting; on the contrary, there was underfitting in this case.

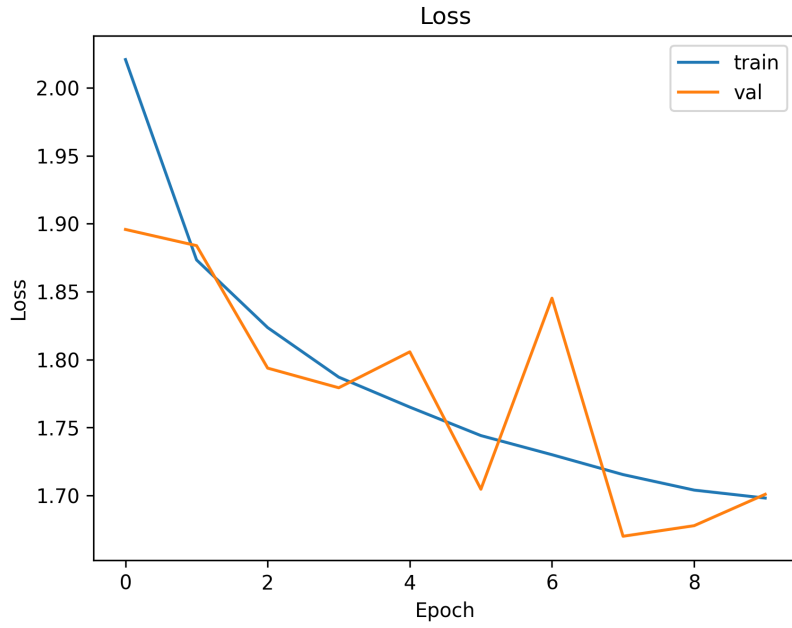


Figure 4: Training and validation loss over epochs (frozen base).

Inference: Loss on training data reduced gradually from 2.0209 to 1.6980, while loss on validation data also reduced gradually to its minimum value of 1.6699 in epoch 8 before plateauing at 1.7008 in epoch 10. This similarity of training and validation losses further validates that the frozen-base model is well-regularized but has not converged yet after 10 epochs.

Task 4: Fine Tuning

The last few convolutional layers of ResNet50 were unfrozen and the model was fine-tuned for 5 additional epochs at a lower learning rate (0.0001) to allow the pretrained features to adapt slightly to CIFAR-10.

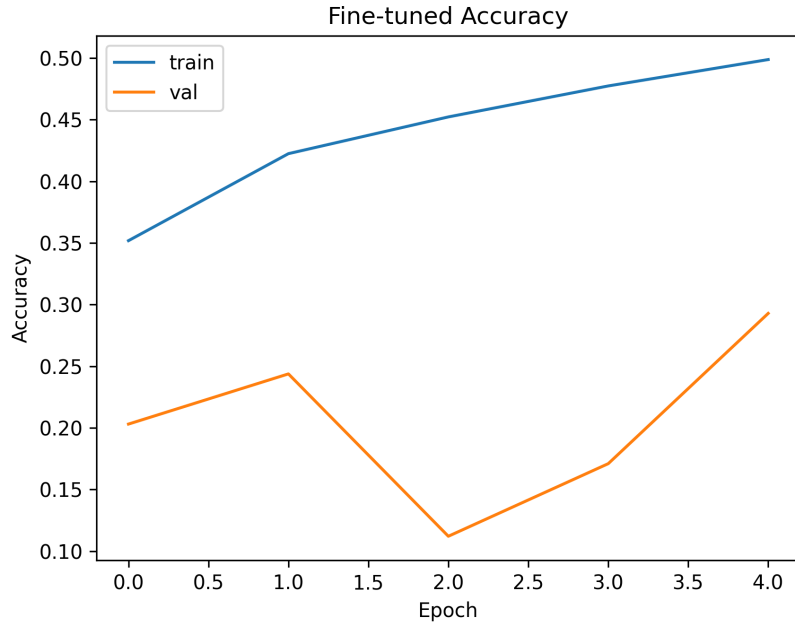


Figure 5: Training and validation accuracy after fine-tuning.

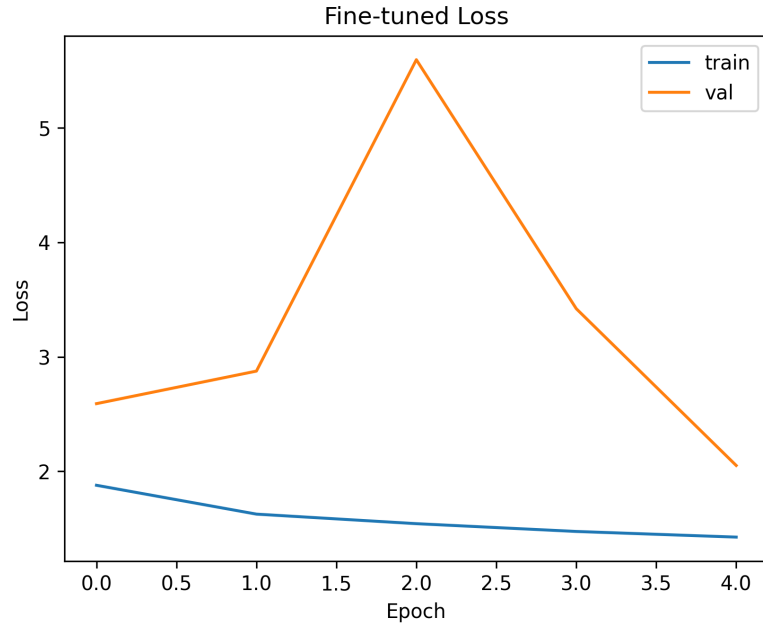


Figure 6: Training and validation loss after fine-tuning.

Inference: There were several positive and negative outcomes from the fine-tuning process. Firstly, training accuracy was significantly increased, going from 35.19% to 49.88% throughout five epochs of fine-tuning, and the training loss decreased from 1.8788 to 1.4262. On the other hand, the validation accuracy became very unstable: it quickly went down from 20.31% (Epoch 1) to as low as 11.21% (Epoch 3) with validation loss reaching 5.5940, and only partially recovered to 29.29% at Epoch 5 (val loss 2.0516).

Such a divergence between stable growing training accuracy and unstable validation accuracy, especially the one which falls at first, indicates a combination of overfitting and training instability issues, most probably caused by unfreezing the convolutional layers (along with BatchNorm statistics) with the learning rate still being too high for newly unfrozen weights, which caused the model to forget some pretrained feature until partially recovering by the final epoch.

Task 5: Model Evaluation

The fine-tuned model was evaluated on the CIFAR-10 test set using accuracy, precision, recall, F1-score, the confusion matrix, and the classification report.

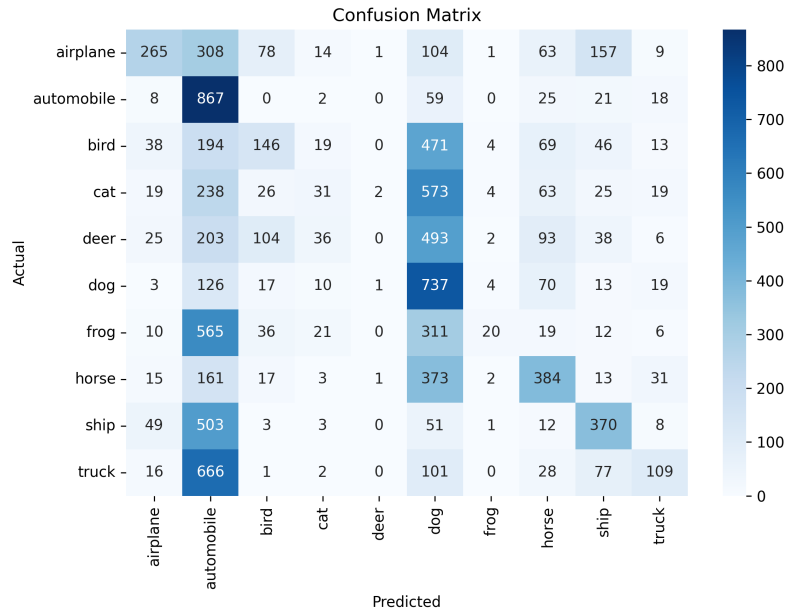


Figure 7: Confusion matrix of the fine-tuned ResNet50 model on the CIFAR-10 test set.

Table 4: Class-wise precision, recall, and F1-score on the CIFAR-10 test set

Class	Precision	Recall	F1-score
Airplane	0.59	0.27	0.37
Automobile	0.23	0.87	0.36
Bird	0.34	0.15	0.20
Cat	0.22	0.03	0.05
Deer	0.00	0.00	0.00
Dog	0.23	0.74	0.34
Frog	0.53	0.02	0.04
Horse	0.46	0.38	0.42
Ship	0.48	0.37	0.42
Truck	0.46	0.11	0.18

Inference: While Ship and Horse had the most balanced results in terms of their F1-score (both 0.42), the class of Deer performed exceptionally poorly with zero precision,

recall and F1 score (model did not identify a single image of a deer). The classes of Automobile and Dog demonstrate a very strong discrepancy between precision and recall (recall values of 0.87 and 0.74 for these two classes, while precision is much lower – 0.23 in both cases), which means that the model over-fitted these two classes and included many instances of other similar classes into these categories (Cat, Deer, trucks). Frog and Cat demonstrate the opposite trend – good precision (0.53 and 0.22), but terrible recall (0.02 and 0.03) – the model did not predict these classes enough times, and therefore almost all predictions were correct. Overall, it is clear that the confusion matrix represents a model that collapsed its predictions into a small number of classes (mainly Automobile and Dog).



Figure 8: Sample misclassified test images with true and predicted labels.

6. Hyperparameter Study

Performance was compared across different hyperparameter settings, as summarized below.

Table 5: Hyperparameter study settings and observations (3-epoch comparisons)

Hyperparameter	Values	Observation
Learning Rate	0.001, 0.0001	0.001 performed slightly better (35.71% vs. 34.68%).
Batch Size	16, 32, 64	Smaller batches performed marginally better (34.12%, 33.51%, 33.30%).
Epochs	10, 20	Accuracy was still increasing at epoch 10, suggesting 20 epochs may improve results.
Optimizer	Adam, SGD	Adam outperformed SGD (32.26% vs. 21.18%) during early training.
Dense Units	128, 256	Performance was nearly identical (32.75% vs. 32.93%), suggesting limited impact.
Frozen Layers	All, Partial	Fully frozen layers performed better (36.46% vs. 22.65%) in this short run.

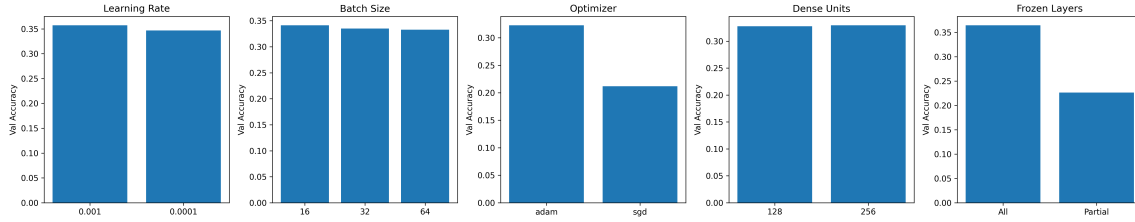


Figure 9: Validation accuracy across different hyperparameter settings (3-epoch quick comparisons).

7. Mandatory Plots

All required plots are included with the corresponding tasks above. For quick reference:

1. Sample CIFAR-10 images – Figure 1
2. Training Accuracy – Figure 3
3. Validation Accuracy – included in Figure 3
4. Training Loss – Figure 4
5. Validation Loss – included in Figure 4
6. Confusion Matrix – Figure 7
7. Misclassified Images – Figure 8

8. Results

8.1 Performance Metrics

Table 6: Final results of the fine-tuned ResNet50 model on CIFAR-10

Metric	Value
Training Accuracy (%)	49.88
Testing Accuracy (%)	29.29
Precision (weighted)	0.35
Recall (weighted)	0.29
F1-score (weighted)	0.24
Training Time	≈ 4.7 min (2.9 min frozen-base + 1.8 min fine-tuning)
Total Parameters	24,114,826 (527,114 trainable + 23,587,712 non-trainable)

Note on overall accuracy: The testing accuracy obtained (29.29%) is noticeably lower than what ResNet50 typically achieves on CIFAR-10 with proper preprocessing. This is primarily because ResNet50 was designed and pretrained on ImageNet images of size 224×224 , whereas CIFAR-10 images are only 32×32 . Feeding 32×32 images directly into ResNet50 (as done here) causes the deep stack of stride-2 convolutions and pooling layers to shrink the feature map to a size of 1×1 or smaller well before the end of the network, destroying most of the spatial information the later residual blocks rely on. This is a

known limitation when applying deep ImageNet-scale architectures directly to small low-resolution datasets, and is consistent with the observed weighted-average precision (0.35) being noticeably higher than recall (0.29), since the model compensated by concentrating its predictions on only a couple of classes (Automobile and Dog) rather than genuinely learning to discriminate all ten.

8.2 Comparison of CNN Architectures

Table 7: Comparison of CNN architectures (reported figures)

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	60K		
AlexNet	61M		
VGG16	138M		
GoogLeNet	6.8M		
ResNet50	25.6M	29.29	≈ 4.7 min

9. Source Code

[GitHub Repository](#)

10. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

11. Discussion

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet was a groundbreaking achievement because it dominated the ImageNet challenge and proved that deep CNNs were better than traditional computer vision techniques. AlexNet employed ReLU, Dropout, GPU training, and data augmentation, which made deep networks easier and faster to train and less likely to suffer from overfitting.

2. Why does VGG16 use only 3×3 convolution filters?

Different 3×3 filters can obtain the same receptive field as the larger filter, using fewer parameters. Moreover, the use of several ReLU layers in the architecture allows the network to learn more complicated features. The use of just one filter size makes the architecture simple.

3. Explain the advantages of the Inception module.

The Inception module uses multiple sizes of filters and pooling simultaneously, allowing detecting features of various sizes. The use of 1×1 convolutions before larger convolutions helps to lower the number of channels, reducing computation and the number of parameters. Thus, the network gains high accuracy with less number of parameters than VGG16.

4. What is the purpose of residual learning?

The main advantage of residual learning is that it makes the deep neural network easier to train by learning the residual $F(x) = H(x) - x$, instead of the complete mapping $H(x)$. The purpose of this method is to overcome the issue of vanishing gradient problem and performance degradation as depth increases.

5. Differentiate LeNet and ResNet.

LeNet-5 is a shallow convolutional neural network with about 60,000 parameters used for grayscale digit images. On the other hand, ResNet50 is a 50-layer deep convolutional neural network with about 25.6 million parameters and uses skip connections to facilitate training of deep neural networks. In conclusion, LeNet introduces a basic CNN model, whereas ResNet enables very deep CNNs.

6. What is Transfer Learning?

In transfer learning, the pretrained network serves as the base for training a network on a similar task. Given the nature of the pretrained network, the first layers already capture the low-level features like edges, textures, etc., the only part that requires retraining is the last few layers.

7. Why is fine tuning required?

A pretrained model was trained on a different dataset, so its features may not perfectly match the new task. Fine-tuning the later layers with a small learning rate allows the model to adapt to the new dataset and usually improves accuracy.

8. Explain the difference between dilated convolution and transpose convolution.

With dilated convolution, the space between kernel units is increased in order to expand the receptive field without increasing parameters and without reducing the resolution of the images. With transpose convolution, the spatial extent of the feature map can be expanded; therefore, it is useful for upsampling and for decoders.

9. Why do pretrained models converge faster?

Pretrained models already have useful features learned from a large dataset. They do not need to learn basic features like edges and textures from scratch, so they usually require fewer training steps and converge faster.

10. Compare the computational complexity of LeNet and ResNet.

LeNet-5 has about 60K parameters and 7 layers, so it is lightweight and computationally cheap. ResNet50 has about 25.6M parameters and 50 layers, making it much more expensive to train and run. However, its greater depth allows it to learn more complex features and achieve much better performance on difficult image classification tasks.

12. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, Gradient-Based Learning Applied to Document Recognition, *Proceedings of the IEEE*, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, ImageNet Classification with Deep Convolutional Neural Networks, *NeurIPS*, 2012.
3. Karen Simonyan and Andrew Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, *ICLR*, 2015.
4. Christian Szegedy et al., Going Deeper with Convolutions, *CVPR*, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, Deep Residual Learning for Image Recognition, *CVPR*, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>